



ARTIFICIAL NEURAL NETWORKS (ANN) LAB

12

Ms. Shakeela Shaheen

LAB 12: INTRODUCTION TO GENERATIVE ADVERSARIAL NETWORKS (GANS)

Learning Objectives

By the end of this lab, students will be able to:

- Understand the concept of **Generative Models**
- Explain the working of **GAN (Generator vs Discriminator)**
- Implement a **basic GAN using PyTorch**
- Train GAN on image dataset
- Generate realistic synthetic images
- Analyze GAN training behavior and challenges

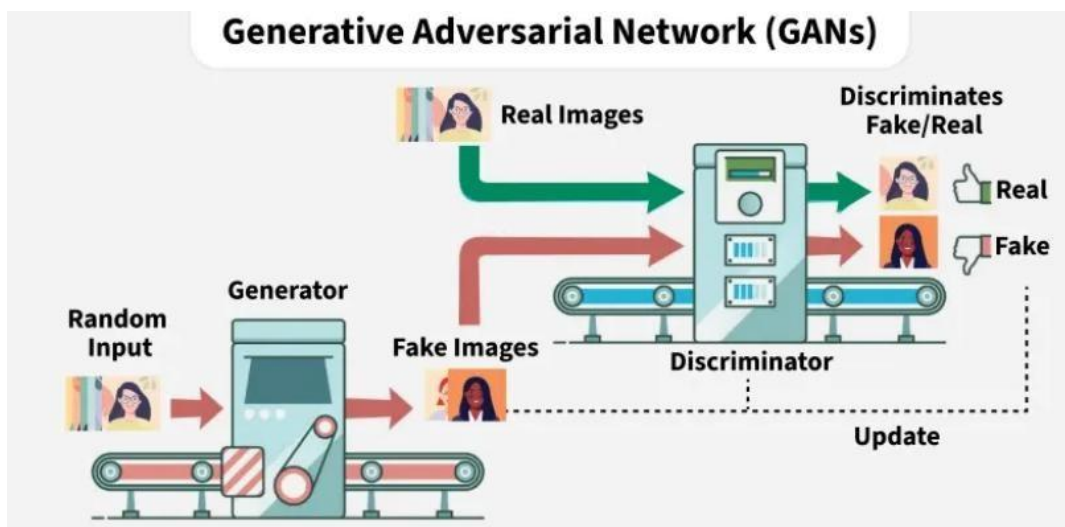
WHY GAN's?

You are working as an **AI Engineer in a medical imaging company**.

The company faces a problem: “There is **limited medical image data** available to train AI models.” What would you do?

That’s where GAN’s come in. GANs are models that generate new, realistic data by learning from existing data. Introduced by Ian Goodfellow in 2014, they enable machines to create content like images, videos and music.

GANs have been called the coolest idea in machine learning in the last twenty years” (Yann LeCun, Chief AI Scientist at Facebook, 2016).



They are useful because:

- Create new data similar to real world data
- Go beyond classification to generate content
- Used in art, gaming, healthcare and data science

What is a GAN?

A **Generative Adversarial Network (GAN)** consists of:

- **Generator (G)** → Creates fake data
- **Discriminator (D)** → Detects fake vs real

Both compete in a **game-like setting**

Core Idea:

Think of it like:

- Generator = **Fake artist**
- Discriminator = **Art critic**

Process:

1. Generator creates fake images
2. Discriminator checks if real or fake
3. Both improve over time

Generator produces data so realistic that discriminator gets confused

GAN Architecture

1. Generator (G)

- Input → Random Noise (z)
- Output → Fake Image

Objective:

Maximize: $D(G(z))$

The generator is a deep neural network that takes random noise as input to generate realistic data samples like images or text. It learns the underlying data patterns by adjusting its internal parameters during training through backpropagation. Its objective is to produce samples that the discriminator classifies as real.

Generator Loss Function: The generator tries to minimize this loss:

$$J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i))$$

where

- J_G measure how well the generator is fooling the discriminator.
- $G(z_i)$ is the generated sample from random noise z_i
- $D(G(z_i))$ is the discriminator's estimated probability that the generated sample is real.

The generator aims to maximize $D(G(z_i))$ meaning it wants the discriminator to classify its fake data as real (probability close to 1).

2. Discriminator (D)

- Input $\rightarrow$ Real + Fake Images
- Output $\rightarrow$ Probability (Real or Fake)

Objective:

$$\text{Maximize: } \log(D(x)) + \log(1 - D(G(z)))$$

The discriminator is a binary classifier that distinguishes real data from generated samples. Through training, it refines its parameters to improve detection of fake data and when working with images, it uses convolutional layers to extract features and enhance classification accuracy.

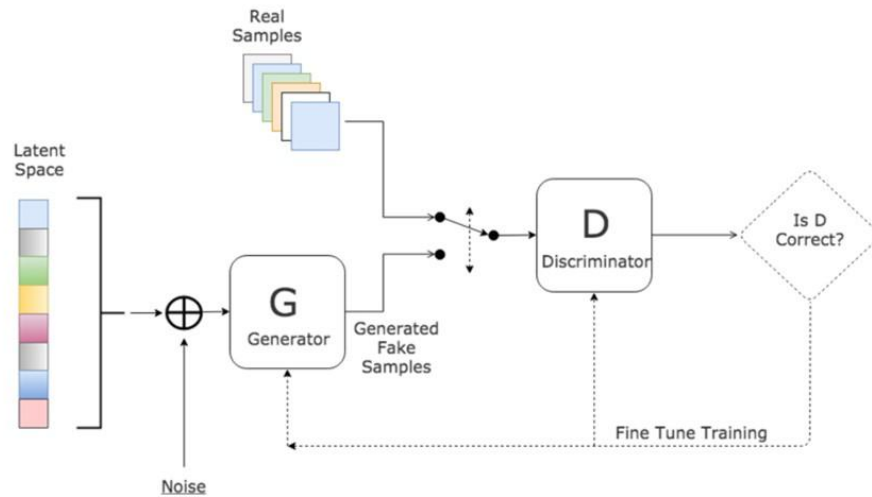
Discriminator Loss Function: The discriminator tries to minimize this loss:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- J_D measures how well the discriminator classifies real and fake samples.
- x_i is a real data sample.
- $G(z_i)$ is a fake sample from the generator.
- $D(x_i)$ is the discriminator's probability that x_i is real.
- $D(G(z_i))$ is the discriminator's probability that the fake sample is real.

The discriminator wants to correctly classify real data as real (maximize $\log D(x_i)$) and fake data as fake (maximize $\log(1 - D(G(z_i)))$)

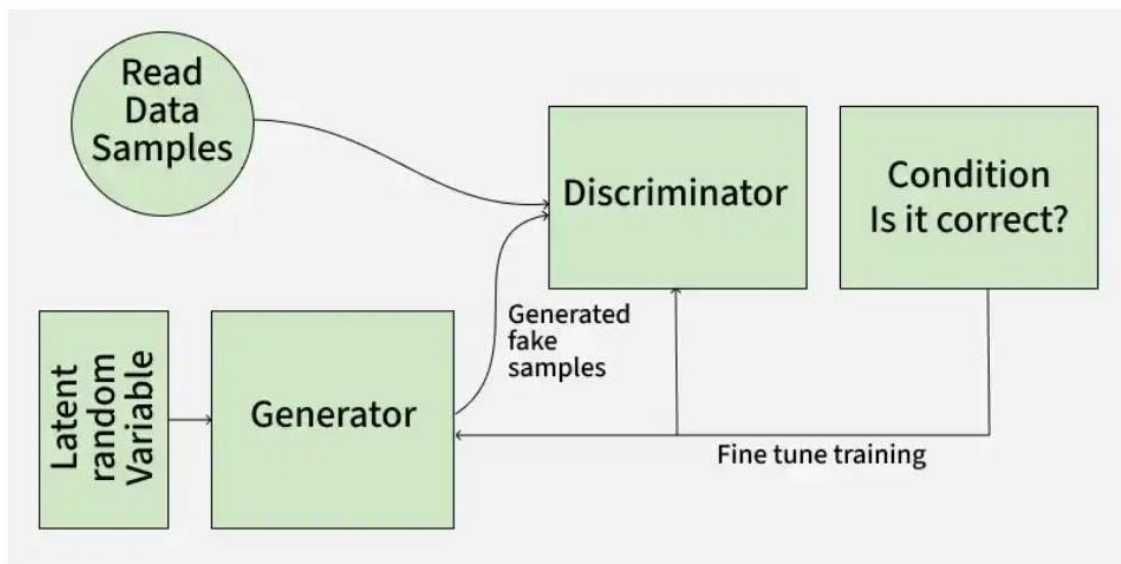
GAN Architecture



[Source: M. Farragher, medium.com, 2019]

$$\min_G \max_D V(G,D)$$

MinMax Loss



GANs are trained using a MinMax Loss between the generator and discriminator:

$$\min_G \max_D V(G, D) = [\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(g(z)))]]$$

where:

- G is generator network and D is the discriminator network
- $p_{data}(x)$ = true data distribution
- $p_z(z)$ = distribution of random noise (usually normal or uniform)
- $D(x)$ = discriminator's estimate of real data
- $D(G(z))$ = discriminator's estimate of generated data

The generator tries to minimize this loss (to fool the discriminator) and the discriminator tries to maximize it (to detect fakes accurately).

GAN TRAINING IN A NUTSHELL:

1. Assemble a batch of real and fake samples:

- Select some real samples
- Produce some fake samples by propagating random inputs through the generator network

2. Propagate the batch through the discriminator network

3. Adapt weights:

- The “error signal” for maximizing the discriminator's error on fake samples is propagated through the discriminator without adapting the weights and further propagated to perform weight updates in the generator.
- The weight updates in the discriminator are done as usual

TECHNICAL ISSUES IN GAN'S:

- The two networks learn simultaneously. In some way, they “bootstrap” each other. In order to ensure that both learn well, their architecture and learning strategies need to fit well. Nonchalantly speaking, one idea is to have slower adaptation in the generator than in the discriminator (cf. Heusel et al., 2018).
- The generator might only create samples from certain subgroups. This issue is called mode collapse. It can be counteracted with special loss functions.

TYPES OF GAN

There are several types of GANs each designed for different purposes. Here are some important types:

1. Vanilla GAN

Vanilla GAN is the simplest type of GAN. It consists of:

- A generator and a discriminator both are built using multi-layer perceptrons (MLPs).
- The model optimizes its mathematical formulation using stochastic gradient descent (SGD).

While foundational, Vanilla GAN can face problems like:

- Generator produces limited types of outputs repeatedly.
- Generator and discriminator may not improve smoothly.

2. Conditional GAN (CGAN)

Conditional GAN (CGAN) adds an additional conditional parameter to guide the generation process. Instead of generating data randomly they allow the model to produce specific types of outputs.

Working of CGANs:

- A conditional variable (y) is fed into both the generator and the discriminator.
- This ensures that the generator creates data corresponding to the given condition (e.g generating images of specific objects).
- The discriminator also receives the labels to help distinguish between real and fake data.

Example: Instead of generating any random image, CGAN can generate a specific object like a dog or a cat based on the label.

3. Deep Convolutional GAN (DCGAN)

Deep Convolutional GAN (DCGAN) are among the most popular types of GANs used for image generation.

They are important because they:

- Uses Convolutional Neural Networks (CNNs) instead of simple multi-layer perceptrons (MLPs).
- Max pooling layers are replaced with convolutional stride helps in making the model more efficient.
- Fully connected layers are removed, which allows for better spatial understanding of images.

DCGANs are successful because they generate high-quality, realistic images.

4. Laplacian Pyramid GAN (LAPGAN)

Laplacian Pyramid GAN (LAPGAN) is designed to generate ultra-high-quality images by using a multi-resolution approach.

Working of LAPGAN:

- Uses multiple generator-discriminator pairs at different levels of the Laplacian pyramid.
- Images are first down sampled at each layer of the pyramid and upscaled again using Conditional GAN (CGAN).
- This process allows the image to gradually refine details and helps in reducing noise and improving clarity.

Due to its ability to generate highly detailed images, LAPGAN is considered a superior approach for photorealistic image generation.

5. Super Resolution GAN (SRGAN)

Super-Resolution GAN (SRGAN) is designed to increase the resolution of low-quality images while preserving details.

Working of SRGAN:

- Uses a deep neural network combined with an adversarial loss function.
- Enhances low-resolution images by adding finer details helps in making them appear sharper and more realistic.
- Helps to reduce common image upscaling errors such as blurriness and pixelation.

IMPLEMENTATION

Generative Adversarial Networks (GAN) can generate realistic images by learning from existing image datasets. Here we will be implementing a GAN trained on the CIFAR-10 dataset using PyTorch.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```


We use PyTorch's transforms to convert images to tensors and normalize pixel values between -1 and 1 for better training stability.

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

Download and load the CIFAR-10 dataset with defined transformations. Use a [DataLoader](#) to process the dataset in mini-batches of size 32 and shuffle the data.

```
train_dataset = datasets.CIFAR10(root='./data', \
    train=True, download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(train_dataset, \
    batch_size=32, shuffle=True)
```

Set important training parameters:

- **latent_dim**: Dimensionality of the noise vector.
- **lr**: Learning rate of the optimizer.
- **beta1, beta2**: Beta parameters for Adam optimizer (e.g 0.5, 0.999)
- **num_epochs**: Number of times the entire dataset will be processed (e.g 10)

```
latent_dim = 100
lr = 0.0002
beta1 = 0.5
beta2 = 0.999
num_epochs = 10
```

Create a neural network that converts random noise into images. Use transpose convolutional layers, batch normalization and [ReLU](#) activations. The final layer uses [Tanh](#) activation to scale outputs to the range [-1, 1].

- **nn.Linear(latent_dim, 128 * 8 * 8)**: Defines a fully connected layer that projects the noise vector into a higher dimensional feature space.
- **nn.Upsample(scale_factor=2)**: Doubles the spatial resolution of the feature maps by upsampling.

- **nn.Conv2d(128, 128, kernel_size=3, padding=1):** Applies a convolutional layer keeping the number of channels the same to refine features.

```
class Generator(nn.Module):
    def __init__(self, latent_dim):
        super(Generator, self).__init__()

        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128 * 8 * 8),
            nn.ReLU(),
            nn.Unflatten(1, (128, 8, 8)),
            nn.Upsample(scale_factor=2),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128, momentum=0.78),
            nn.ReLU(),
            nn.Upsample(scale_factor=2),
            nn.Conv2d(128, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64, momentum=0.78),
            nn.ReLU(),
            nn.Conv2d(64, 3, kernel_size=3, padding=1),
            nn.Tanh()
        )

    def forward(self, z):
        img = self.model(z)
        return img
```

Create a binary classifier network that distinguishes real from fake images. Use convolutional layers, batch normalization, dropout, LeakyReLU activation and a Sigmoid output layer to give a probability between 0 and 1.

- **nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1):** Second convolutional layer increasing channels to 64, downsampling further.
- **nn.BatchNorm2d(256, momentum=0.8):** Batch normalization for 256 feature maps with momentum 0.8.

```

class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()

        self.model = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.25),
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
            nn.ZeroPad2d((0, 1, 0, 1)),
            nn.BatchNorm2d(64, momentum=0.82),
            nn.LeakyReLU(0.25),
            nn.Dropout(0.25),
            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(128, momentum=0.82),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.25),
            nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(256, momentum=0.8),
            nn.LeakyReLU(0.25),
            nn.Dropout(0.25),
            nn.Flatten(),
            nn.Linear(256 * 5 * 5, 1),
            nn.Sigmoid()
        )

    def forward(self, img):
        validity = self.model(img)
        return validity

```

- **Generator and Discriminator** are initialized on the available device (GPU or CPU).
- **Binary Cross-Entropy (BCE) Loss** is chosen as the loss function.
- **Adam optimizers** are defined separately for the generator and discriminator with specified learning rates and betas.

```

generator = Generator(latent_dim).to(device)
discriminator = Discriminator().to(device)

adversarial_loss = nn.BCELoss()

optimizer_G = optim.Adam(generator.parameters()\
                           , lr=lr, betas=(beta1, beta2))
optimizer_D = optim.Adam(discriminator.parameters()\
                           , lr=lr, betas=(beta1, beta2))

```

Train the discriminator on real and fake images, then update the generator to improve its fake image quality. Track losses and visualize generated images after each epoch.

- **valid = torch.ones(real_images.size(0), 1, device=device):** Create a tensor of ones representing real labels for the discriminator.
- **fake = torch.zeros(real_images.size(0), 1, device=device):** Create a tensor of zeros representing fake labels for the discriminator.
- **z = torch.randn(real_images.size(0), latent_dim, device=device):** Generate random noise vectors as input for the generator.
- **g_loss = adversarial_loss(discriminator(gen_images), valid):** Calculate generator loss based on the discriminator classifying fake images as real.
- **grid = torchvision.utils.make_grid(generated, nrow=4, normalize=True):** Arrange generated images into a grid for display, normalizing pixel values.

```
for epoch in range(num_epochs):
    for i, batch in enumerate(dataloader):

        real_images = batch[0].to(device)

        valid = torch.ones(real_images.size(0), 1, device=device)
        fake = torch.zeros(real_images.size(0), 1, device=device)

        real_images = real_images.to(device)

        optimizer_D.zero_grad()

        z = torch.randn(real_images.size(0), latent_dim, device=device)

        fake_images = generator(z)

        real_loss = adversarial_loss(discriminator\
                                    (real_images), valid)
        fake_loss = adversarial_loss(discriminator\
                                    (fake_images.detach()), fake)
        d_loss = (real_loss + fake_loss) / 2

        d_loss.backward()
        optimizer_D.step()

        optimizer_G.zero_grad()

        gen_images = generator(z)

        g_loss = adversarial_loss(discriminator(gen_images), valid)
```

```

gen_images = generator(z)

g_loss = adversarial_loss(discriminator(gen_images), valid)
g_loss.backward()
optimizer_G.step()

if (i + 1) % 100 == 0:
    print(
        f"Epoch [{epoch+1}/{num_epochs}]\n"
        f"Batch {i+1}/{len(dataloader)} "
        f"Discriminator Loss: {d_loss.item():.4f} "
        f"Generator Loss: {g_loss.item():.4f}"
    )
if (epoch + 1) % 10 == 0:
    with torch.no_grad():
        z = torch.randn(16, latent_dim, device=device)
        generated = generator(z).detach().cpu()
        grid = torchvision.utils.make_grid(generated, \
                                           nrow=4, normalize=True)

        plt.imshow(np.transpose(grid, (1, 2, 0)))
        plt.axis("off")
        plt.show()

```



Output

By following these steps we successfully implemented and trained a GAN that learns to generate realistic CIFAR-10 images through adversarial training.

LAB TASK

Problem Statement

A startup is building an AI system for **handwritten digit recognition**, but:

- Dataset is limited
- Model overfits

Your task: Use **GAN** to generate synthetic handwritten digits

Dataset (Kaggle)

Use:

— MNIST Digit Recognizer Dataset

<https://www.kaggle.com/datasets/oddrationalale/mnist-in-csv>

Task 1: Data Preparation

- Load dataset
- Normalize images

Task 2: GAN Implementation

- Implement Generator
- Implement Discriminator

Task 3: Training

- Train GAN for 10–20 epochs
- Observe losses

Task 4: Visualization

- Generate fake images
- Compare with real images

Task 5: Experimentation

Try:

- Change latent dimension ($100 \rightarrow 50 \rightarrow 200$)
- Change learning rate
- Add more layers

Expected Output

- Generated handwritten digits
- Loss graphs (Generator vs Discriminator)
- Visual comparison

Lab Questions (To be Answered in Notebook Cells)

1. Why are GANs called “adversarial”?
2. What happens if discriminator becomes too strong?
3. What is mode collapse?
4. Why do we use random noise as input?
5. Difference between GAN and CNN?